Creating Join Tables

(https://github.com/learn-co-curriculum/python-p3-creating-join-tables) (https://github.com/learn-co-curriculum/python-p3-creating-join-tables/issues/new)

Learning Goals

- Use primary and foreign keys to create the relations in a relational database.
- Use `INNER JOIN` and `LEFT JOIN` to retrieve relevant data for members of one table in other tables.
- Use joins to create one-to-many and many-to-many relationships between tables.

Key Vocab

- **Primary Key**: a number that uniquely identifies one record in a table.
- **Foreign Key**: a column or group of columns that connects one table to another.
- **Join**: a query that returns related records from multiple tables in a single record.
- **One-to-Many**: a type of relationship between tables where one record in table A is connected to multiple records in table B. **e.g.** One person ordering multiple drinks at a bar.
- **Many-to-Many**: a type of relationship between tables where multiple records in table A are connected to multiple records in table B. **e.g.** Students have many classes and classes have many students.

Introduction

As programmers, we understand the data we are manipulating to be real. In other words, we write programs to solve real-world problems or handle real-world situations. Whether you're developing a web application that helps doctors and hospitals around the country manage patient information or coding a command line game, the code we write is modeled on real situations and things. This is true of our databases and tables as well as code we write in Python, Ruby, you name it.

We use databases to store information that represents real-world entities. We might have an employee database in which an individual row in an `employees` table represents one real, individual employee. Such a database might also have a `managers` table that is related to the `employees` table.

Real-world objects and environments are relational. Employees belong to managers, pets belong to owners, a person has many friends. Our relational databases have the ability to reflect that related-ness.

In relational databases, we can actually categorize the type of relationship that exists between the data that we are storing. There are two basic types of relationship that we need to concern ourselves with right now: the **one-to-many** relationship and the **many-to-many** relationship. Let's take a closer look.

One-To-Many Relationships

Let's continue using our previous pets database as an example. This pets database has an `owners` table and a `cats` table. The `cats` table has a column, `owner_id`, that contains a foreign key corresponding to the `id` column of the `owners` table.

In this way, an individual cat is associated to the person that owns it. Any number of cats can have the same owner. In other words, any number of cats can have the same `owner_id`.

Let's say we have the following owners:

```
id              name
--------------  ----------
1               mugumogu
2               Sophie
3               Penny
```

And the following cats:

```
id              name        age         owner_id    breed
--------------  ----------  ----------  ----------  -------------
1               Maru        3           1           Scottish Fold
```

```
2               Hana        1           1           Tabby
3               Nona        4           2           Tortoiseshell
4               Lil' Bub    2                       perma-kitten
```

Note that both Maru and Hana have the same value in the `owner_id` column, a value of `1`. This corresponds to the owner from the `owners` table who has an `id` of `1`. Both Maru and Hana, therefore, have the same owner: mugumogu. If we run a query to select all of the cats whose `owner_id` is `1`, we'll return both Maru and Hana.

The following query:

```
sqlite> SELECT * FROM cats WHERE owner_id = 1;
```

...returns:

```
id              name        age         owner_id    breed
--------------  ----------  ----------  ----------  -------------
1               Maru        3           1           Scottish Fold
2               Hana        1           1           Tabby
```

Our first owner, mugumogu, *has many* cats. Both Hana and Maru *belong to* mugumogu. This is the **one-to-many**, or a "has many"/"belongs to", relationship.

Enacting The Relationship Through Foreign Keys

The **one-to-many** relationship is created through the use of foreign keys. The `cats` table has an `owner_id` column which is the foreign key column. It contains information that corresponds to the `id` column of the `owners` table.

To put it another way, **one** owner has **many** cats.

The table that contains the foreign key column is the table that contains the entities that **belong to** another entity. The table that is referenced via the foreign key is the owner entity that **has many** of something else. This relationship works because multiple entities in the "belonging" or child table can have the same foreign key.

What happens, though, when a cat realizes it can live the good life by hanging out with the family across the street for extra food and care? Such a cat would have *more than one owner*. Our one-to-many relationship is no longer adequate.

How might we account for a cat with many owners? Well, we could continue to add additional `owner_id` columns to the cats table. For example we could add an `owner_id_1`, `owner_id_2`, `owner_id_3` column and so on. This is not practical however. It requires us to change our schema by continuing to add columns every time a cat gains a new owner. This means our `cats` table could grow to contain a possibly infinite number of columns (some cats are very popular, after all).

We can avoid this undesirable horizontal table growth with the use of a **join table**.

Join Tables and the Many-to-Many Relationship

A **join table** contains common fields from two or more other tables. In this way, it creates a **many-to-many** relationship between data. Let's take a closer look at this concept by building our own join table in the following code-along.

Code Along 1: Building a Join Table

We want to create a many-to-many association between cats and owners, such that a cat can have many owners and an owner can have many cats. Our join table will therefore have two columns, one for each of the tables we want to relate. We will have a `cat_id` column and an `owner_id` column.

Here's what the Entity Relationship Diagram (ERD) will look like for our database:

Let's set up our database to get started.

Setting Up the Database

In your terminal, create the `pets_database.db` file by running the following command:

```
sqlite3 pets_database.db
```

Create the following two tables:

Cats Table:

```
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER,
  breed TEXT
);
```

Owners Table:

```
CREATE TABLE owners (
  id INTEGER PRIMARY KEY,
  name TEXT
);
```

Insert the following data:

Insert Data:

```
INSERT INTO owners (name) VALUES ("mugumogu");
INSERT INTO owners (name) VALUES ("Sophie");
INSERT INTO owners (name) VALUES ("Penny");
INSERT INTO cats (name, age, breed) VALUES ("Maru", 3, "Scottish Fold");
INSERT INTO cats (name, age, breed) VALUES ("Hana", 1, "Tabby");
INSERT INTO cats (name, age, breed) VALUES ("Nona", 4, "Tortoiseshell");
INSERT INTO cats (name, age, breed) VALUES ("Lil' Bub", 2, "perma-kitten");
```

The `cat_owners` Join Table

Creating the Table

Now we're ready to create our join table. Since our table is creating a **many-to-many** relationship between cats and owners, we will call our table `cat_owners`. It is conventional to name your join tables using the names of the tables you are creating the many-to-many relationship between.

Execute the following SQL statement to create our join table:

```
CREATE TABLE cat_owners (
  cat_id INTEGER,
  owner_id INTEGER
);
```

Now we're ready to start inserting some rows into our join table.

Inserting Data into the Join Table

Each row in our join table will represent one cat/owner relationship. Let's say, for example, that Nona the cat has acquired a second owner, Penny. Now we want to represent that Nona has two owners, Sophie and Penny.

First, we'll insert the Nona/Sophie relationship into our join table. Recall that Nona the cat has an `id` of `3` and Sophie the owner has an `id` of `2`.

```
INSERT INTO cat_owners (cat_id, owner_id) VALUES (3, 2);
```

Now let's check the contents of our `cat_owners` table with a SELECT statement:

```
SELECT * FROM cat_owners;
```

This should return:

```
cat_id          owner_id
--------------  ----------
3               2
```

Now let's insert the Nona/Penny relationship into our join table:

```
INSERT INTO cat_owners (cat_id, owner_id) VALUES (3, 3);
```

We'll confirm this insertion with another SELECT statement:

```
SELECT * FROM cat_owners;
```

This should return:

```
cat_id          owner_id
--------------  ----------
3               2
3               3
```

Now our table reflects that Nona, the cat with an `id` of `3`, has many (in this case two) owners.

The great thing about our join table, however, is that it allows for the **many-to-many** relationship. We have a cat with many owners!

As you may have deduced, our **many-to-many** relationship shares many of the same characteristics as our **one-to-many** relationship: it still uses foreign keys and primary keys to connect between two different tables. A **many-to-many** relationship is essentially two one-to-many relationships that go through a common join table: to get from a cat to all of its owners, we must go **through** the `cat_owners` table. We could describe that relationship like this:

A cat **has many** owners **through** the `cat_owners` table.

Next, let's insert a row that will give a particular owner many cats.

Sophie's dream has come true and now she is a co-owner of Maru the cat. Let's insert the appropriate row into our join table. Remember that Sophie has an `id` of `2` and Maru has an `id` of `1`. Let's insert that row:

```
INSERT INTO cat_owners (cat_id, owner_id) VALUES (1, 2);
```

Let's run a SELECT statement to confirm that our insertion worked:

```
SELECT * FROM cat_owners;
```

This should return:

```
cat_id          owner_id
--------------  ----------
3               2
3               3
1               2
```

Nona, our cat with an `id` of `3` has many owners and Sophie, our owner with an `id` of `2`, has many cats. Our many-to-many relationship is up and running.

As we can see, just like a cat can have many owners through the `cat_owners` table, it is also true that an owner **has many** cats **through** the `cat_owners` table.

Now let's take advantage of this association by running some queries that utilize our join table to return information about these complex relationships.

Code Along 2: Querying the Join Table

Basic Queries

Let's SELECT from our join table all of the owners who are associated to cat number 3.

```
SELECT cat_owners.owner_id
FROM cat_owners
WHERE cat_id = 3;
```

This should return:

```
owner_id
--------------
```

```
2
3
```

Now let's SELECT all of the cats who are associated with owner number 2:

```
SELECT cat_owners.cat_id
FROM cat_owners
WHERE owner_id = 2;
```

That should return:

```
cat_id
---------------
3
1
```

These queries are great, but it would be even better if we could write queries that would return us some further information about the cats and owners we are returning here, such as their names. Otherwise it becomes a little difficult to constantly remember cats and owners by ID only. We can do so by querying our join tables using JOIN statements.

Advanced Queries

Execute the following query:

```
SELECT owners.name
FROM owners
INNER JOIN cat_owners
ON owners.id = cat_owners.owner_id WHERE cat_owners.cat_id = 3;
```

This should return:

```
name
---------------
```

```
Sophie
Penny
```

Let's break down the above query:

- `SELECT owners.name` : Here, we declare the column data that we want to actually have returned to us.
- `FROM owners` : Here, we specify the table whose column we are querying.
- `INNER JOIN cat_owners ON owners.id = cat_owners.owner_id` : Here, we are joining the `cat_owners` table on the `owners` table. We are telling our query to look for owners whose `id` column matches up to the `owner_id` column in the `cat_owners` table.
- `WHERE cat_owners.cat_id = 3;` : Here, we are adding an additional condition to our query. We are telling our query to look at the `cat_owners` table rows where the value of the `cat_id` column is `3` . Then, *for those rows only*, cross reference the `owner_id` column value with the `id` column in the `owners` table.

Let's take a look at a boiler-plate query that utilizes a JOIN statement to query a join table:

```
SELECT column(s)
FROM table_one
INNER JOIN table_two
ON table_one.column_name = table_two.column_name
WHERE table_two.column_name = condition;
```

Giving this one more try, let's query the join table for the names of all of the cats owned by Sophie:

```
SELECT cats.name
FROM cats
INNER JOIN cat_owners
ON cats.id = cat_owners.cat_id
WHERE cat_owners.owner_id = 2;
```

This should return:

```
name
---------------
```

```
Nona
Maru
```

We can also join our third table by expanding the query, again following the same pattern using primary and foreign keys:

```
SELECT
  cats.name AS cat_name,
  owners.name AS owner_name
FROM cats
INNER JOIN cat_owners
  ON cats.id = cat_owners.cat_id
INNER JOIN owners
  ON cat_owners.owner_id = owners.id;
```

Which will return information about both the cats and owners:

```
cat_name    owner_name
----------  ----------
Nona        Sophie
Nona        Penny
Maru        Sophie
```

Conclusion

You've now seen how to create two of the most common relationships in SQL: the **one-to-many**, also known as the "has many"/"belongs to" relationship, and the **many-to-many**, or "has many through" relationship. Both of these relationships are built around primary and foreign keys. The difference is that in order to create a **many-to-many** relationship, you will need to create *multiple* one to many relationships using a **join table**, as we did in the example above.

Resources

- Difference between Primary Key and Foreign Key - GeeksforGeeks (https://www.geeksforgeeks.org/difference-between-primary-key-and-foreign-key/)
- SQL Joins - W3Schools (https://www.w3schools.com/sql/sql_join.asp)
- Airtable's guide to many-to-many relationships (https://support.airtable.com/hc/en-us/articles/218734758-Airtable-s-guide-to-many-to-many-relationships)
- Practice SQL Queries on SQLBolt (http://sqlbolt.com/lesson/select_queries_review)